Task A3 – Discover the Data-Access Design Pattern (5 Marks, K4)

Part A asks for design, pseudocode and reasoning, so the answer is in pseudocode and tables. The Java in the *Alternative* sections is only for illustration. Items marked `<fill>` need your own observations.

A3.1 – Identify the common pattern

Step	Same in StudentDB, CourseDB, MarksDB?	Part that differs
1. Open a connection (`DriverManager.getConnection`)	Same	–
2. Build the SQL string	Same step	SQL text (table and column names)
3. Create a statement	Same	–
4. Bind parameters	Same step	which parameters and their types
5. Execute (`executeQuery` / `executeUpdate`)	Same	–
6. Read the `ResultSet` and build objects	Same step	which columns map to which fields of `Student`, `Course`, `Marks`
7. Close ResultSet, statement and connection (try-with-resources)	Same	–
8. Handle `SQLException`	Same	–

This is the **Data Access Object (DAO) pattern**, together with a *template method*: the repeated steps (1, 3, 5, 7, 8) are written once and only SQL, parameter binding and row mapping are supplied per entity.

A3.2 – Generic data-access interface

```
public interface Dao<T, K> {
    void insert(T entity) throws SQLException;
    T findById(K id) throws SQLException;          // null if not found
    List<T> findAll() throws SQLException;
    void update(T entity) throws SQLException;
    void delete(K id) throws SQLException;
}
```

The shared steps are written once in a small helper (`JdbcTemplate`, with `update(sql, binder)` and `query(sql, binder, rowMapper)`), and `StudentDao implements Dao<Student, String>` only supplies SQL, binding and mapping. The same shape gives `CourseDao implements Dao<Course, String>` and `MarksDao implements Dao<Marks, MarksKey>`.

```
import java.sql.SQLException;
import java.util.List;

/** Generic data-access interface: T = entity type, K = primary-key
type. */
public interface Dao<T, K> {
    void insert(T entity) throws SQLException;
    T findById(K id) throws SQLException;          // null if not
found
    List<T> findAll() throws SQLException;
    void update(T entity) throws SQLException;
    void delete(K id) throws SQLException;
}
```

```
import java.sql.*;
import java.util.*;

/** The repeated steps (open, prepare, bind, execute, map, close)
written once. */
public class JdbcTemplate {
    public interface Binder { void bind(PreparedStatement ps) throws
SQLException; }
    public interface RowMapper<T> { T map(ResultSet rs) throws
SQLException; }

    private final String url, user, pwd;
    public JdbcTemplate(String url, String user, String pwd) {
this.url = url; this.user = user; this.pwd = pwd; }

    public int update(String sql, Binder b) throws SQLException {
        try (Connection c = DriverManager.getConnection(url, user,
pwd);
             PreparedStatement ps = c.prepareStatement(sql)) {
            b.bind(ps);
            return ps.executeUpdate();
        }
    }
    public <T> List<T> query(String sql, Binder b, RowMapper<T> m)
throws SQLException {
        List<T> out = new ArrayList<>();
        try (Connection c = DriverManager.getConnection(url, user,
pwd);
             PreparedStatement ps = c.prepareStatement(sql)) {
            b.bind(ps);
```

```
            try (ResultSet rs = ps.executeQuery()) { while (rs.next())
out.add(m.map(rs)); }
        }
        return out;
    }
}

import java.sql.*;
import java.util.List;

/** Only the parts that differ (SQL text, binding, row mapping) live
here. */
public class StudentDao implements Dao<Student, String> {
    private final JdbcTemplate t;
    public StudentDao(JdbcTemplate t) { this.t = t; }

    private static Student map(ResultSet rs) throws SQLException {
        return new Student(rs.getString("reg_no"),
rs.getString("name"),
                           rs.getString("department"),
rs.getInt("study_year"));
    }
    @Override public void insert(Student s) throws SQLException {
        t.update("INSERT INTO student(reg_no, name, department,
study_year) VALUES (?,?,?,?)", ps -> {
            ps.setString(1, s.getRegNo()); ps.setString(2,
s.getName());
            ps.setString(3, s.getDepartment()); ps.setInt(4,
s.getYear()); });
    }
    @Override public Student findById(String id) throws SQLException {
        List<Student> r = t.query("SELECT * FROM student WHERE reg_no
= ?", ps -> ps.setString(1, id), StudentDao::map);
        return r.isEmpty() ? null : r.get(0);
    }
    @Override public List<Student> findAll() throws SQLException {
        return t.query("SELECT * FROM student ORDER BY reg_no", ps ->
{}, StudentDao::map);
    }
    @Override public void update(Student s) throws SQLException {
        t.update("UPDATE student SET name=?, department=?,
study_year=? WHERE reg_no=?", ps -> {
            ps.setString(1, s.getName()); ps.setString(2,
s.getDepartment());
            ps.setInt(3, s.getYear()); ps.setString(4, s.getRegNo());
});
    }
    @Override public void delete(String id) throws SQLException {
        t.update("DELETE FROM student WHERE reg_no = ?", ps ->
ps.setString(1, id));
```

```
    }
}
```

A3.3 – Map objects to tables

```
class Student { String regNo; String name; String department; int
year; }
```

```
CREATE TABLE student (
    reg_no     VARCHAR(12) PRIMARY KEY,     -- Student.regNo
(primary key)
    name       VARCHAR(50) NOT NULL,        -- Student.name
    department VARCHAR(30) NOT NULL,        -- Student.department
    study_year INT NOT NULL                 -- Student.year  (YEAR is
a reserved word, so a different column name is used)
);
```

Java field	Type	Column	SQL type	Notes
`regNo`	`String`	`reg_no`	`VARCHAR(12)`	**primary key**
`name`	`String`	`name`	`VARCHAR(50)`	
`department`	`String`	`department`	`VARCHAR(30)`	
`year`	`int`	`study_year`	`INT`	field and column names differ; the mapping lives only in `StudentDao`

Alternative concept and complexity

SQL written directly in the service or GUI classes (no generic `Dao`): every screen and service that touches `student` contains its own `getConnection`, SQL text and `ResultSet` loop.

- **Places to change when a column is renamed (e.g. `department` → `dept`):** with `StudentDao`, **1 place** (the SQL strings and `map` method in that class, plus the schema). With SQL scattered in n service/GUI classes, **n places** (every class that mentions the column), and a missed one fails only at run time. If the number of classes using the table is n, the change cost is O(1) versus O(n).
- **Time and space per database operation:** essentially the same. Both designs send the same SQL to the same database, so the cost of the query itself (parsing, I/O) is identical. The DAO adds one extra method call and one small lambda object per call: O(1) time and space, negligible compared with a network round trip. The generic design is a maintainability and correctness win, not a performance cost. (Both open a connection per call as written here; for performance use a connection pool in either design.)